#Part A

```
import tensorflow as tf
from keras.datasets import fashion_mnist
import matplotlib.pyplot as plt
import cv2
import numpy as np
import pandas as pd
```

Loading datasets

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-labels-idx1-ubyte.gz
32768/29515 [==============================] - 0s 0us/step
40960/29515 [==========================================] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-images-idx3-ubyte.gz
26427392/26421880 [==============================] - 0s 0us/step
26435584/26421880 [==============================] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-labels-idx1-ubyte.gz
16384/5148
[==============================================================================
==========================] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-images-idx3-ubyte.gz
4423680/4422102 [==============================] - 0s 0us/step
4431872/4422102 [==============================] - 0s 0us/step
```

```
type(x_train)
```

```
numpy.ndarray
```

```
x_train[0].shape,y_train[0].shape,x_test[0].shape,y_test.shape
```

```
((28, 28), (), (28, 28), (10000,))
```

Plot grayscale images

```
def plot_images(img):

  plt.figure(figsize=(10,4))
  plt.imshow(img, cmap = 'gray')
```

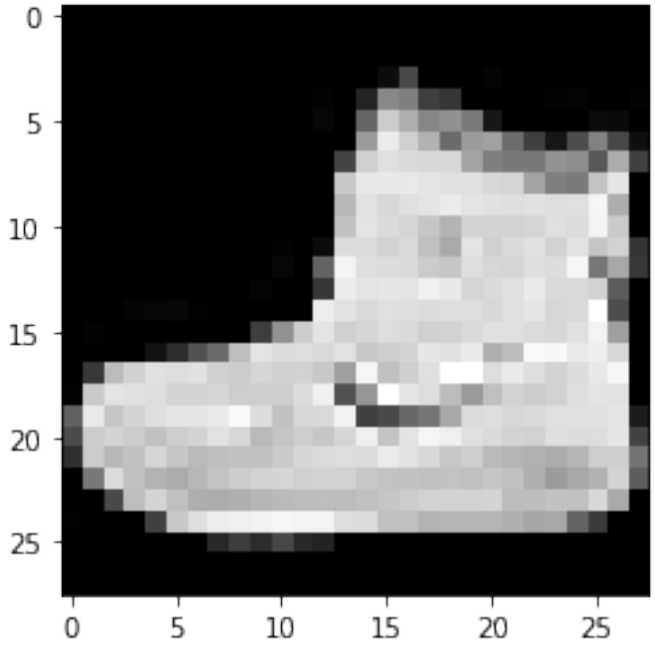
Flip the image

```
def mirror(img):
  image = cv2.flip(img,1)
  return image

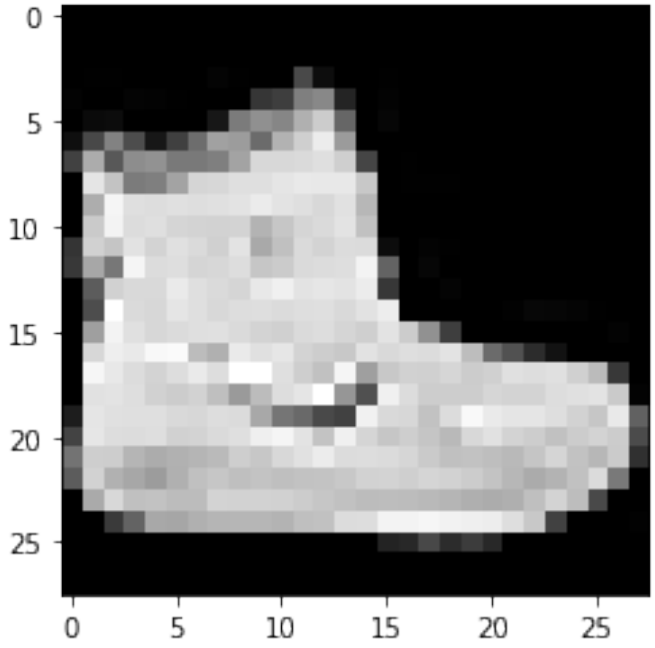
img = mirror(x_train[0])
img.shape
```

```
(28, 28)
```

```
plot_images(x_train[0])
```

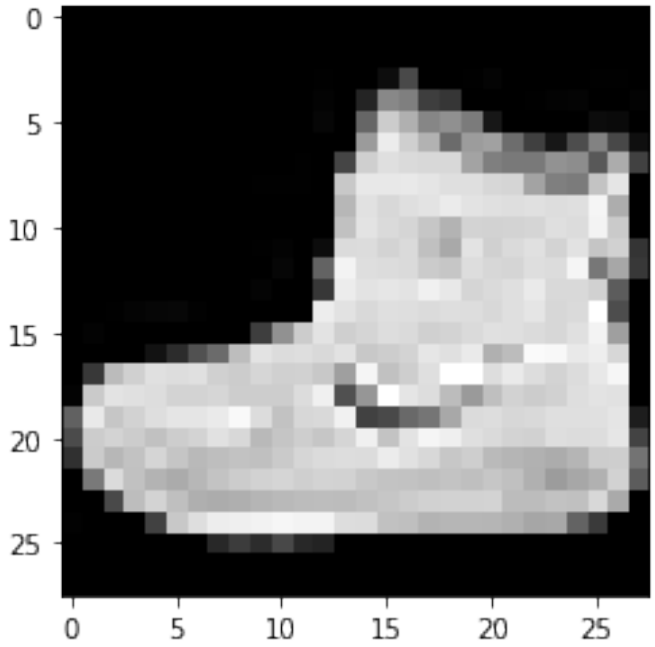


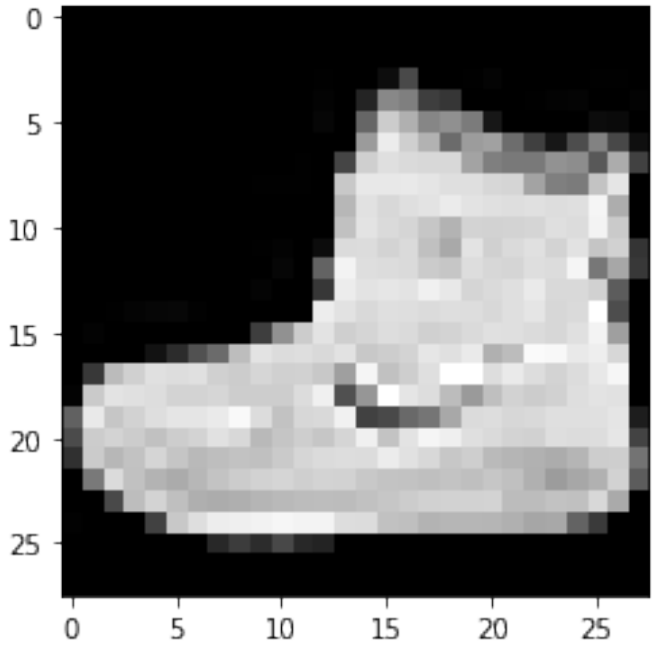

```
plot_images(img)
```




```
plot_images(x_train[0]/255),plot_images(x_train[0])
```

```
(None, None)
```






```
x_train= x_train/255.0
x_test=x_test/255.0

y_train.shape
```

```
(60000,)
```

```
y_train = y_train.reshape(-1,1)
y_train.shape
```

```
(60000, 1)
```

```
y_test = y_test.reshape(-1,1)
y_test.shape
```

```
(10000, 1)
```

flipping image

```
x_train_mirror = []
for i in x_train:
  x_train_mirror.append(mirror(i))
x_train_mirror = np.array(x_train_mirror)
x_train_mirror.shape
```

```
(60000, 28, 28)
```

```
x_test_mirror = []
for i in x_test:
  x_test_mirror.append(mirror(i))
x_test_mirror = np.array(x_test_mirror)
x_test_mirror.shape
```

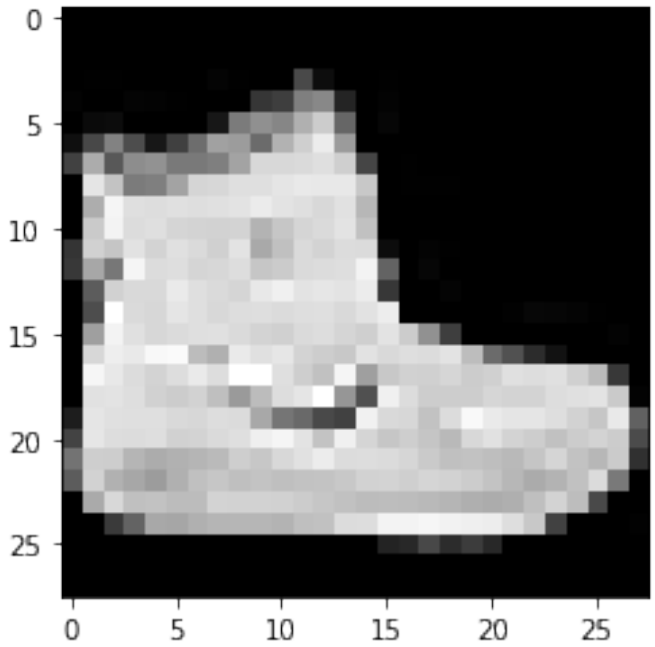
```
(10000, 28, 28)
```

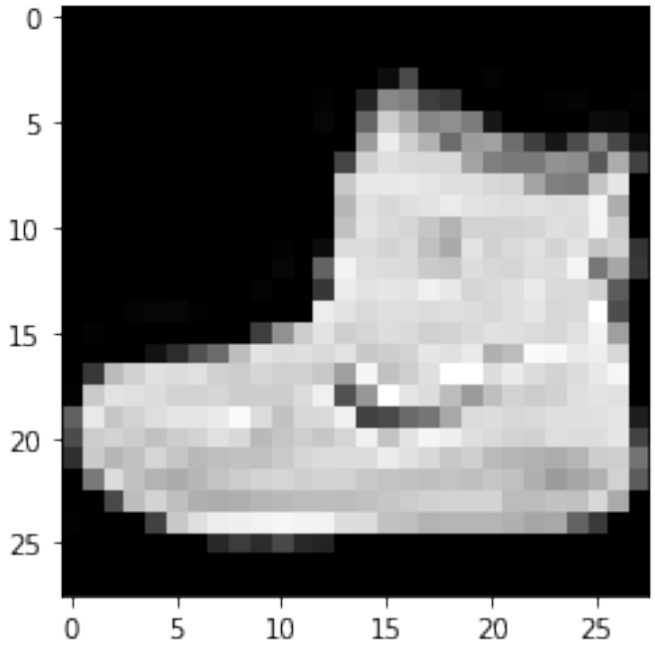
```
y_train[0]
```

```
array([9], dtype=uint8)
```

```
plot_images(x_train_mirror[0]),plot_images(x_train[0])
```

```
(None, None)
```






```
d = {
0: 'T-shirt/top',
1 : 'Trouser',
2 : 'Pullover',
3 : 'Dress',
4 : 'Coat',
5 : 'Sandal',
6 : 'Shirt',
7 : 'Sneaker',
8 : 'Bag',
9 : 'Ankle boot'
}
```

```
x_train_mod = np.vstack((x_train,x_train_mirror))
x_train_mod.shape
```

```
(120000, 28, 28)
```

```
y_train_mod = np.vstack((y_train,y_train))
y_train_mod.shape
```

```
(120000, 1)
```

```
y_train_mod[1],y_train_mod[60001]
```

```
(array([0], dtype=uint8), array([0], dtype=uint8))
```

```
x_test_mod = np.vstack((x_test,x_test_mirror))
x_test_mod.shape
```

```
(20000, 28, 28)
```

```
y_test_mod = np.vstack((y_test,y_test))
y_test_mod.shape
```

```
(20000, 1)
```

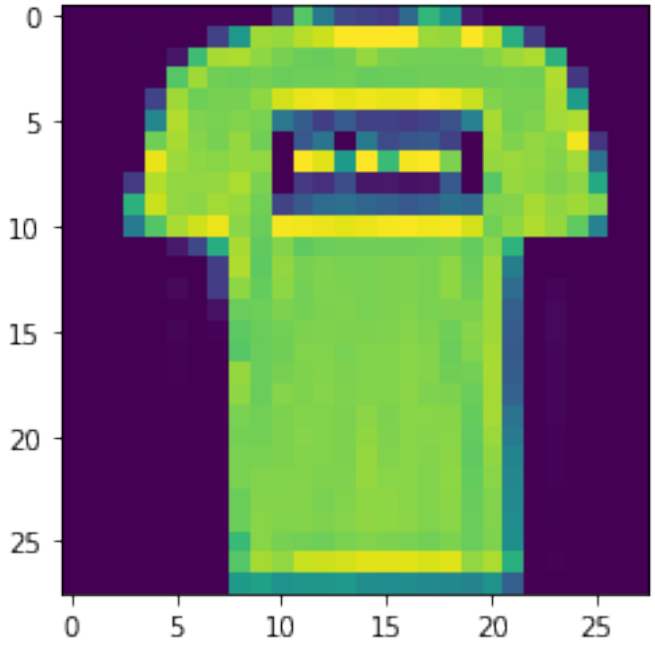
```
y_test_mod[1],y_test_mod[10001]
```

```
(array([2], dtype=uint8), array([2], dtype=uint8))
```

```
def plot_color_images(img):

  plt.figure(figsize=(10,4))
  plt.imshow(img)

plot_color_images(x_train_mod[1])
```

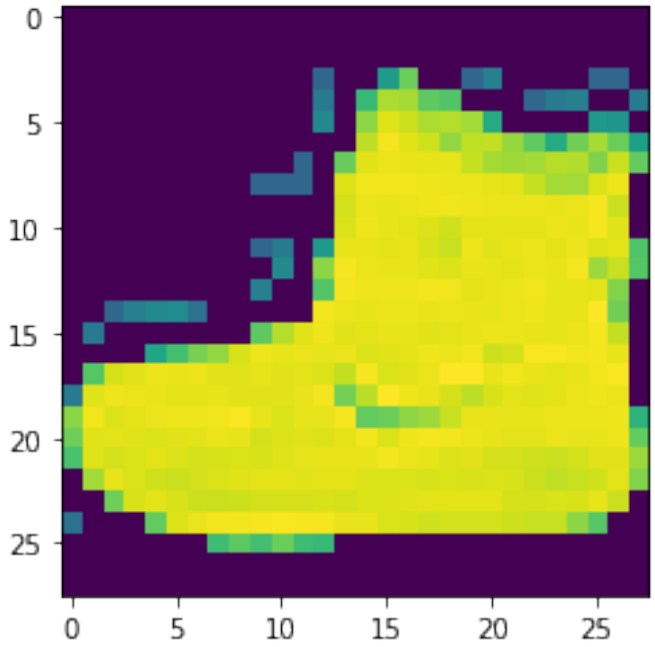


Power law transform

```
def power_law(img):
  gamma = 0.2
  gamma_corrected = np.array(255*(img) ** gamma, dtype = 'uint8')
  #plot_images(gamma_corrected)
  #plot_color_images(gamma_corrected)
  #print(gamma_corrected)
  return gamma_corrected

img = power_law(x_train_mod[0])

plot_color_images(img)
```



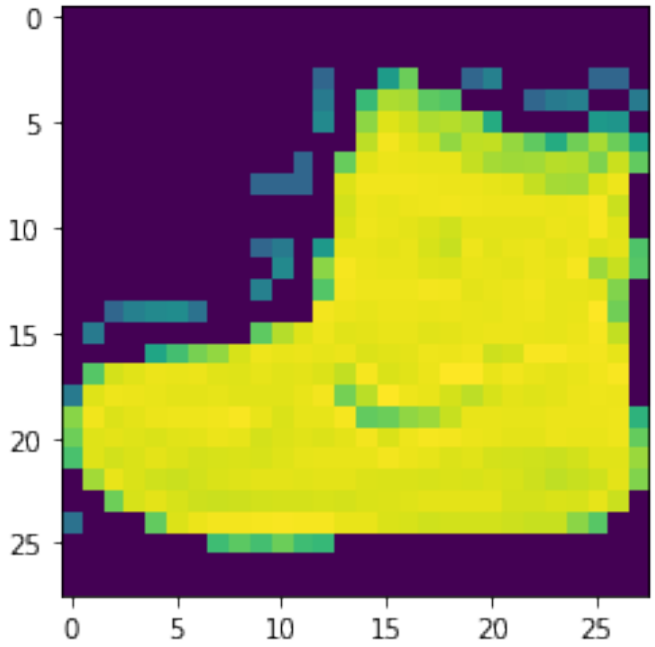

```
x_train_gamma = np.array(255*(x_train) ** 0.2, dtype = 'uint8')
x_train_gamma.shape
```

```
(60000, 28, 28)
```

```
x_test_gamma = np.array(255*(x_test) ** 0.2, dtype = 'uint8')
x_test_gamma.shape
```

```
(10000, 28, 28)
```

```
plot_color_images(x_train_gamma[0])
```




```
x_train_mod = np.vstack((x_train_mod,x_train_gamma))
x_train_mod.shape
```

```
(180000, 28, 28)
```

```
x_test_mod = np.vstack((x_test_mod,x_test_gamma))
x_test_mod.shape
```

```
(30000, 28, 28)
```

```
y_train_mod = np.vstack((y_train_mod,y_train))
y_train_mod.shape
```

```
(180000, 1)
```

```
y_test_mod = np.vstack((y_test_mod,y_test))
y_test_mod.shape
```

```
(30000, 1)
```

```
x_train_mod.shape,y_train_mod.shape,x_test_mod.shape,y_test_mod.shape
```

```
((180000, 28, 28), (180000, 1), (30000, 28, 28), (30000, 1))
```

```
x_train = x_train_mod.copy()
x_train.shape
```

```
(180000, 28, 28)
```

```
y_train = y_train_mod.copy()
y_train.shape
```

```
(180000, 1)
```

```
x_val = x_test_mod[15000:]
x_test = x_test_mod[:15000]
x_val.shape,x_test.shape
```

```
((15000, 28, 28), (15000, 28, 28))
```

```
y_val = y_test_mod[15000:]
y_test = y_test_mod[:15000]
y_val.shape,y_test.shape
```

```
((15000, 1), (15000, 1))
```

#Part B

Loading Libraries

```
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense,
Flatten
```

```
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)
x_test = x_test.reshape(x_test.shape[0], 28, 28, 1)
```

```
y_train = y_train.reshape(180000)
y_test = y_test.reshape(-1)
y_val = y_val.reshape(-1)
y_test.shape
```

```
(15000,)
```

convolution neural network

```
num_filters = 64
filter_size1 = 3
pool_size1 = 2
```

```
model1 = Sequential()
model1.add(Conv2D(filters=num_filters, kernel_size=filter_size1,
input_shape=(28,28,1),padding='same',activation = 'relu'))
model1.add(MaxPooling2D(pool_size = pool_size1))
model1.add(Conv2D(filters=num_filters, kernel_size=filter_size1,
padding='same',activation = 'relu'))
model1.add(MaxPooling2D(pool_size = pool_size1))
model1.add(Flatten())
model2.add(Dense(256, activation='relu'))
model1.add(Dense(10,activation = 'softmax'))
```

```
model1.summary()
```

```
Model: "sequential_12"
_________________________________________________________________
```

```
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_24 (Conv2D)          (None, 28, 28, 64)        640

 max_pooling2d_24 (MaxPoolin  (None, 14, 14, 64)       0
 g2D)

 conv2d_25 (Conv2D)          (None, 14, 14, 64)        36928

 max_pooling2d_25 (MaxPoolin  (None, 7, 7, 64)         0
 g2D)

 flatten_12 (Flatten)        (None, 3136)              0

 dense_15 (Dense)            (None, 10)                31370

=================================================================
Total params: 68,938
Trainable params: 68,938
Non-trainable params: 0
_________________________________________________________________
```

```
model1.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

model1.fit(x_train, to_categorical(y_train), epochs = 5, verbose = 1,
validation_data = (x_val, to_categorical(y_val

)))
```

```
Epoch 1/5
5625/5625 [==============================] - 42s 7ms/step - loss:
0.5565 - accuracy: 0.8115 - val_loss: 0.4130 - val_accuracy: 0.8487
Epoch 2/5
5625/5625 [==============================] - 41s 7ms/step - loss:
0.3604 - accuracy: 0.8690 - val_loss: 0.3798 - val_accuracy: 0.8631
Epoch 3/5
5625/5625 [==============================] - 41s 7ms/step - loss:
0.3197 - accuracy: 0.8822 - val_loss: 0.3713 - val_accuracy: 0.8634
Epoch 4/5
5625/5625 [==============================] - 42s 7ms/step - loss:
0.2991 - accuracy: 0.8895 - val_loss: 0.3767 - val_accuracy: 0.8625
Epoch 5/5
5625/5625 [==============================] - 42s 7ms/step - loss:
0.2876 - accuracy: 0.8936 - val_loss: 0.3546 - val_accuracy: 0.8731

<keras.callbacks.History at 0x7f4907e82d10>
```

```
loss1,score1 = model1.evaluate(x_test, to_categorical(y_test),
verbose=0)
```

```
print("Loss: ",loss1)
print('Accuracy', score1)
```

```
Loss:  0.29241740703582764
Accuracy 0.8939333558082581
```

```
predictions = model1.predict(x_test[:10])
print(np.argmax(predictions, axis = 1))
```

```
[9 2 1 1 6 1 4 6 5 7]
```

```
y_test[:10]
```

```
array([9, 2, 1, 1, 6, 1, 4, 6, 5, 7], dtype=uint8)
```

#Part C

```
from keras.layers import BatchNormalization,Dropout
```

Dropout regularization

```
model2 = Sequential()
model2.add(Conv2D(filters=num_filters, kernel_size=filter_size1,
input_shape=(28,28,1),padding='same',activation = 'relu'))
model2.add(MaxPooling2D(pool_size = pool_size1))
model2.add(Dropout(0.2))
model2.add(Conv2D(filters=num_filters, kernel_size=filter_size1,
padding='same',activation = 'relu'))
model2.add(MaxPooling2D(pool_size = pool_size1))
model2.add(Dropout(0.2))
model2.add(Flatten())
model2.add(Dense(256, activation='relu'))
model2.add(Dropout(0.2))
model2.add(Dense(10,activation = 'softmax'))
```

```
model2.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

```
model2.fit(x_train, to_categorical(y_train), epochs = 5, verbose = 1,
validation_data = (x_val, to_categorical(y_val

)))
```

```
Epoch 1/5
5625/5625 [==============================] - 51s 9ms/step - loss:
0.6427 - accuracy: 0.7818 - val_loss: 0.4416 - val_accuracy: 0.8301
Epoch 2/5
5625/5625 [==============================] - 51s 9ms/step - loss:
0.4052 - accuracy: 0.8500 - val_loss: 0.4237 - val_accuracy: 0.8457
Epoch 3/5
5625/5625 [==============================] - 50s 9ms/step - loss:
0.3635 - accuracy: 0.8652 - val_loss: 0.3480 - val_accuracy: 0.8667
```

```
Epoch 4/5
5625/5625 [==============================] - 51s 9ms/step - loss:
0.3469 - accuracy: 0.8714 - val_loss: 0.3365 - val_accuracy: 0.8773
Epoch 5/5
5625/5625 [==============================] - 50s 9ms/step - loss:
0.3347 - accuracy: 0.8769 - val_loss: 0.3328 - val_accuracy: 0.8779

<keras.callbacks.History at 0x7f49080985d0>

model2.summary()

Model: "sequential_11"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_22 (Conv2D)          (None, 28, 28, 64)        640

 max_pooling2d_22 (MaxPoolin  (None, 14, 14, 64)       0
 g2D)

 dropout_8 (Dropout)         (None, 14, 14, 64)        0

 conv2d_23 (Conv2D)          (None, 14, 14, 64)        36928

 max_pooling2d_23 (MaxPoolin  (None, 7, 7, 64)         0
 g2D)

 dropout_9 (Dropout)         (None, 7, 7, 64)          0

 flatten_11 (Flatten)        (None, 3136)              0

 dense_12 (Dense)            (None, 256)               803072

 dropout_10 (Dropout)        (None, 256)               0

 dense_13 (Dense)            (None, 10)                2570

=================================================================
Total params: 843,210
Trainable params: 843,210
Non-trainable params: 0
_________________________________________________________________

loss2,score2 = model2.evaluate(x_test, to_categorical(y_test),
verbose=0)
print("Loss: ",loss2)
print('Accuracy', score2)

Loss:  0.2769969701766968
Accuracy 0.897266685962677
```

Batch Normalization

```
model3 = Sequential()

model3.add(Conv2D(filters=num_filters, kernel_size=filter_size1,
input_shape=(28,28,1),padding='same',activation = 'relu'))
model3.add(BatchNormalization())
model3.add(MaxPooling2D(pool_size = pool_size1))
model3.add(Conv2D(filters=num_filters, kernel_size=filter_size1,
padding='same',activation = 'relu'))
model3.add(BatchNormalization())
model3.add(MaxPooling2D(pool_size = pool_size1))
model3.add(Flatten())
model2.add(Dense(256, activation='relu'))
model3.add(BatchNormalization())
model3.add(Dense(10,activation = 'softmax'))

model3.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

model3.fit(x_train, to_categorical(y_train), epochs = 5, verbose = 1,
validation_data = (x_val, to_categorical(y_val

)))
```

```
Epoch 1/5
5625/5625 [==============================] - 53s 9ms/step - loss:
0.6042 - accuracy: 0.8016 - val_loss: 0.6514 - val_accuracy: 0.8207
Epoch 2/5
5625/5625 [==============================] - 51s 9ms/step - loss:
0.4330 - accuracy: 0.8519 - val_loss: 0.4689 - val_accuracy: 0.8565
Epoch 3/5
5625/5625 [==============================] - 52s 9ms/step - loss:
0.3611 - accuracy: 0.8692 - val_loss: 0.3961 - val_accuracy: 0.8582
Epoch 4/5
5625/5625 [==============================] - 51s 9ms/step - loss:
0.3322 - accuracy: 0.8783 - val_loss: 0.3592 - val_accuracy: 0.8686
Epoch 5/5
5625/5625 [==============================] - 52s 9ms/step - loss:
0.3154 - accuracy: 0.8836 - val_loss: 0.3682 - val_accuracy: 0.8687

<keras.callbacks.History at 0x7f4907c4a050>
```

```
model3.summary()
```

```
Model: "sequential_13"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
```

```
 conv2d_26 (Conv2D)          (None, 28, 28, 64)        640

 batch_normalization_6 (Batc  (None, 28, 28, 64)       256
 hNormalization)

 max_pooling2d_26 (MaxPoolin  (None, 14, 14, 64)       0
 g2D)

 conv2d_27 (Conv2D)          (None, 14, 14, 64)        36928

 batch_normalization_7 (Batc  (None, 14, 14, 64)       256
 hNormalization)

 max_pooling2d_27 (MaxPoolin  (None, 7, 7, 64)         0
 g2D)

 flatten_13 (Flatten)        (None, 3136)              0

 batch_normalization_8 (Batc  (None, 3136)             12544
 hNormalization)

 dense_17 (Dense)            (None, 10)                31370

=================================================================
Total params: 81,994
Trainable params: 75,466
Non-trainable params: 6,528
_________________________________________________________________
```

```
loss3,score3 = model3.evaluate(x_test, to_categorical(y_test),
verbose=0)
print("Loss: ",loss3)
print('Accuracy', score3)
```

```
Loss:  0.3133034110069275
Accuracy 0.8871999979019165
```